

Register Method Documentation

1. Contract Table

Item	Description
API Name	Register User API
HTTP Method	POST
Route	/api/register
Purpose	Register a new user account
Request Headers	Content-Type: application/json
Request Body	{ "name": "John", "email": "john@example.com", "password": "Pass@123" }
Required Fields	name, email, password
Password Rule	Minimum 8 characters and at least 1 special character
Success Status Code	201 Created
Validation Error Code	400 Bad Request
Duplicate User Code	409 Conflict
Server Error Code	500 Internal Server Error
Response Format	JSON
Success Response Example	{ "success": true, "message": "User registered successfully" }
Validation Error Example	{ "success": false, "error": "Invalid password format" }
Duplicate User Example	{ "success": false, "error": "Email already exists" }
Data Source	users.json
Business Logic	Validate input → Check duplicate email → Hash password → Save new user

2. Sequence Diagram (Request / Response Map)

Client

|

| POST /api/register

| { name, email, password }

v

Express Route (/api/register)

|

| Validate required fields

| Validate password format

|

| Read users.json

v

File System (fs.readFile)

|

| Return existing users

v

Express Route

|

| Check duplicate email

|

| Hash password

|

| Add new user object

|

| Save updated users.json

v

File System (fs.writeFile)

|

| Save completed

v

Express Route

|

| Return success response

v

Client

|

| 201 Created

| { success: true, message: "User registered successfully" }

3. GenAI Prompt

Prompt

Create an Express.js register API using Express Router.

Requirements:

1. Create a POST route at `/api/register`.

2. Accept JSON request body with:

- name

- email

- password

3. Validate all required fields.

4. Validate password rules:

- Minimum 8 characters

- At least one special character

5. Read existing users from `users.json` using `fs.promises.readFile`.

6. Parse JSON safely.

7. Check if email already exists.

8. If email already exists:

- Return status 409.

- Return:

{

success: false,

error: "Email already exists"

}

9. Hash the password before saving.

10. Create a new user object.

11. Save updated users back to `users.json` using `fs.promises.writeFile`.

12. Return success response:

{

success: true,

message: "User registered successfully"

}

13. Use async/await.

14. Handle server errors with:

{

success: false,

error: "Internal Server Error"